Milestone Four Enhancement Three: Databases

Zain Cronk

Southern New Hampshire University

CS-499 Computer Science Capstone

Instructor Jeff Sanford

October 4, 2026

The artifact selected for this enhancement is an Android inventory management application that I originally developed in CS 360: Mobile Architecture and Programming. The application was created using Java in Android Studio and uses SQLite for persistent storage of user and inventory data. The application allows users to create and manage inventory records, including adding, editing, deleting, searching, and sorting inventory items. This version of the artifact also incorporates the improvements completed during my previous software design and algorithms and data structures enhancements. For the database enhancement, I expanded the application's existing inventory data structure and database functionality to support an additional category attribute for inventory records and improved the security of stored user credentials.

I selected this artifact for the database category because the application relies on SQLite to persist and manage both inventory and user data. The original application provided basic database functionality for storing inventory records, while this enhancement allowed me to demonstrate how an existing database can be expanded without losing previously stored data. I added a category attribute to inventory records and updated the database schema, data model, and create, read, and update operations to support the additional data.

A significant improvement was implementing database versioning and migration. Rather than deleting and recreating the database when the schema changed, I increased the database version and used the onUpgrade() method to modify existing databases. The initial migration added the new category column while preserving existing inventory records. I also updated the application's add and edit functionality so category data can be entered, modified, retrieved from SQLite, and displayed with each inventory item.

I further enhanced the database by improving how user passwords are stored. The original application stored passwords directly in the users table. I replaced this design with

password hashes and unique salts generated for each account. Passwords are now processed using PBKDF2 with HMAC-SHA256 before being stored in SQLite. Authentication retrieves the stored hash and salt, hashes the password entered by the user using the same parameters, and compares the resulting hash with the stored value rather than querying the database using a plaintext password.

This security enhancement also required another database migration. Existing databases could contain user accounts using the original plaintext password column, so simply changing the users table definition would have made those accounts incompatible with the new authentication process. I expanded the onUpgrade() process to create the new users table structure, migrate existing accounts by hashing their existing passwords with newly generated salts, remove the original table, and replace it with the updated table. This allowed existing credentials to remain usable while eliminating the plaintext password field from the upgraded database.

I expanded the automated testing to verify the new database functionality. Tests confirm that category data is stored and updated correctly and that an existing database can be upgraded without losing its inventory records. Additional authentication tests verify that newly created accounts contain password hashes and salts, that valid credentials are accepted, and that incorrect passwords are rejected. I also expanded the database migration test to simulate an older database containing both inventory records and user credentials and verify that the data remains usable after upgrading to the current schema. The completed instrumentation test suite passed all tests.

The database enhancement met the course outcome I planned to address by demonstrating my ability to use well-founded computing techniques, skills, and tools to implement a solution that provides value and accomplishes project goals. Expanding the SQLite

database required changes to the database schema, application data model, database operations, and user interface while maintaining the existing functionality of the application. Implementing database versioning and migration also allowed the application to support the new category data without removing existing inventory records.

The password-storage enhancement also provided additional evidence toward the course outcome related to developing a security mindset. Examining the existing database design identified plaintext password storage as a vulnerability that was not necessary for the application to function. Replacing plaintext credentials with salted password hashes reduced the exposure of user credentials if the database were accessed by an unauthorized party. The migration also required consideration of how to improve security without making existing user accounts unusable. Based on the completed enhancement, I do not currently have any other changes to my planned outcome coverage.

Enhancing the database demonstrated that even a relatively small change to a persistent data structure can affect many parts of an application. Adding a category to an inventory record required more than modifying the SQLite table. The change also had to be incorporated into the application's data model, database operations, add and edit interfaces, data retrieval, and inventory display. Similarly, changing password storage affected database creation, authentication, account creation, schema migration, and testing. This helped me better understand the relationship between persistent storage and the application components that consume and modify that data.

One of the more important challenges was determining how to modify the database without losing or invalidating existing data. Simply changing the table creation statements would work for a new installation but would not update an existing database. I addressed this by

increasing the database version and implementing sequential migrations in onUpgrade(). The category enhancement could be handled by adding a column, while the password-storage enhancement required a more substantial migration because the structure of the users table changed. This required creating a replacement table, transforming existing credential data, and then replacing the original table.

Testing the migration exposed an additional challenge. An existing migration test originally simulated an older database containing only the inventory table because that was sufficient for the previous category migration. After adding the user credential migration, upgrading that simulated database also executed the password migration and failed because the test database did not contain the original users table. I updated the test fixture to more accurately represent the earlier database schema, including both inventory and user data. The revised test then verified that inventory records remained intact and that an existing user could still authenticate after the complete database upgrade. This demonstrated the importance of ensuring that migration tests accurately represent the database versions they are intended to simulate.

During development, I also considered additional improvements that could be made to the database design. Categories could be expanded into a separate table and selected from predefined values, which would improve normalization and data consistency. I also considered how database migrations would need to be handled in a production environment. Although the migrations implemented for this enhancement preserve and transform existing records, a more robust implementation could create a backup of the existing database before performing an upgrade and provide a recovery method if the migration fails. Both improvements were beyond the requirements of this enhancement, so I chose to keep the implementation appropriate for the scope of the project. Considering these alternatives helped me evaluate the trade-offs between

additional normalization, migration reliability, security, and the complexity appropriate for the application.